

Same File In, Same Bytes Out: Reproducible Conduct Records for Agent-Facing Services, Tested by an Independent Reimplementation

Working paper, v0.3, 2026-09-06. Not yet posted.

Toshikatsu Oga, The HORIZ 音s株式会社 (HORIZON SHIELD), Hiratsuka, Japan. ORCID 0009-0000-9180-903X.

Federico Blanco Sánchez-Llanos, Viper Labs (builds invinoveritas, a verification layer for autonomous agents).

Division of work, stated for the record: T.O. designed and operates NENRIN, wrote the reference implementation, and drafted this paper. F.B.S.L. wrote the independent Node.js implementation, performed the blind runs, and wrote Sections 4.2 and 6, including Table 3. Both authors approve the final text before posting.

Abstract

AI agents can already discover services. What they cannot obtain is a record of how a service behaved that the service did not author. Scores and rankings fill that gap today, and scores are gameable by construction. NENRIN is a public, Bitcoin-anchored record of conduct for agent-facing services (Model Context Protocol servers and similar endpoints) built on counts rather than scores. Its third layer, the ring, bundles one calendar month of measured history for one endpoint into one file: how many times it was sampled, how many times it answered, how many distinct tool surfaces were observed, how many independent witnesses measured it, and what the record cannot say. The specification claims that anyone holding the same history can rebuild the ring byte for byte. This paper tests that claim the hard way. A second implementer, who had executed the reference builder as a black box but had not read its body, wrote a builder in a different language (Node.js) from the anchored specification and the published ring files, and ran it against the eight published August 2026 rings. All eight reproduced byte for byte, and both results were anchored (JIDEC entries 32 and 34). The exercise surfaced one seam that a naive port would have crossed without noticing: the specification does not state the canonical form at all, so nested key ordering was inherited from a language runtime and reached the second implementer through the published artifacts rather than through the specification. We report the protocol, the eight matches, the seam and its class, and the limits: byte reproducibility proves that the ring layer is deterministic, not that the measurements inside it are true; two implementations are not a community; the August rings carry a single witness. A blind rerun against the September rings, which will be the first to include a second witness, is pre-registered here.

1. The question

A claim of the form “anyone can verify this” costs nothing to make and is rarely tested by anyone but its author. In the growing ecosystem of agent-facing services, verification claims are common and tests are not. This paper is about one such claim and one such test.

The claim is in the NENRIN specification, Layer 3: a ring is deterministic in its inputs, so a stranger holding the same measured history can rebuild the ring and obtain the same bytes, and therefore the same SHA-256 that was anchored to Bitcoin. Until 2026-09-05 that claim had been exercised by exactly one person, the author of the reference implementation, which is to say it had not been tested at all. Running one’s own program twice is not reproducibility.

The test is an independent reimplementation. A second person reads the specification, checks that the specification he is reading is the one the reference cites (by hash), writes a builder in a language the reference does not use, and runs it blind against the published inputs. Either the bytes match, or they do not, and either result is a finding. This is the discipline of reproducible builds applied not to software artifacts but to behavioral measurements of live services.

We report a match, eight of eight, and we report what the match does and does not establish.

2. Background

2.1 Discovery without conduct

Agent-facing services are discovered through registries and directories. Discovery is solved; conduct is not recorded. In one measured thirty-day window, one MCP server appeared in 93,983 search results and was invoked zero times. What an agent, or a person deciding whether to let an agent connect, cannot obtain is a record taken by parties other than the service itself, accumulated over time, and published whether or not it flatters the subject.

The common substitutes are scores, rankings and badges. Each is a single number chosen by whoever publishes it, and each can be improved by acting on the number rather than on the conduct. NENRIN’s design rule is counts with denominators, never rates,

scores or ranks, so that any reader who computes a rate has visibly left the record.

2.2 Prior art

The components are known and are named deliberately, in the specification and here. Transparency logs (Certificate Transparency; Sigstore Rekor) provide append-only, publicly auditable records. Reproducible builds (the Reproducible Builds project; Debian's bit-for-bit reproducibility effort) establish that independently produced artifacts should be byte-identical from the same source. Supply-chain provenance (in-toto; SLSA) records who did what to which inputs. Anchored timestamps (OpenTimestamps) bind a hash to a Bitcoin block. JSON canonicalization (RFC 8785, the JSON Canonicalization Scheme) defines a language-neutral serialization so that the same data structure yields the same bytes in any implementation. None of these is claimed as new.

What NENRIN combines, and what this paper tests one layer of, is: conduct of agent-facing services as the subject; an open witness model where anyone may submit anchored observations; witness disagreement as a first-class citable record; chained monthly bundles that form a duration credential no one can purchase; and the operator's own services measured under identical rules with failures retained. The specification states this combination as a falsifiable novelty claim and invites anyone who finds a prior instance to submit it as a witness record. This paper does not repeat that claim; it tests the reproducibility of one layer.

3. NENRIN Layer 3 as implemented

3.1 Inputs

A ring is built from two inputs. The first is the history export of one endpoint from the measuring instrument (the verification gate), a JSON document of dated records, each carrying a record SHA-256, a status (verified, pending, held), a consent source, and where measured, the hash of the endpoint's tool manifest. The second, optional, input is a set of witness records: walks of the same endpoint submitted by independent parties to the ledger in the jidec-path-v1 format, each carrying a walked_at time, a base URL, node fetches with body hashes, assertions, a verdict, and a witness identity.

A practical failure is worth recording. The instrument originally retained thirty records per endpoint. By early September the August window for the busiest endpoints was one sweep away from falling out of retention, which would have made the August ring unbuildable from a live export. Retention was raised to four hundred and, more importantly, the history exports were placed under an append-only daily archive in a public repository (github.com/ogasurfproject-jpg/mcp-conduct-register, directory history/). The rings in this paper were built from, and were reproduced from, those committed exports. A record that can only be rebuilt from a live endpoint is not reproducible; a record rebuilt from committed bytes is.

3.2 Output

For one endpoint and one calendar month, the builder emits one JSON object with these fields: schema (nenrin-ring-v1), ring (the month), endpoint, first_instant, last_instant, instants_sampled, instants_reached, instants_by_status, instants_by_consent_source, manifest_hashes_observed, surface_changes, witnesses, witness_identities, discrepancies, record_sha256_first, record_sha256_last, prev_ring, prev_ring_sha256, limits, and recompute. Every numeric field is a count. The limits field is a sentence generated from the counts that states what the ring cannot say, for example that with one witness no discrepancy could have been recorded, or that determinism was not measured on some instants because no owner consent existed to call a tool.

3.3 Bytes

The ring file is the UTF-8 encoding of the object serialized with keys sorted at every nesting level, two-space indentation, non-ASCII characters unescaped, and a single trailing newline. The reference implementation expresses this as Python's `json.dumps(obj, ensure_ascii=False, sort_keys=True, indent=2)` followed by a newline. The specification itself does not state this form; at version 1 it lives only in the reference builder's docstring and the ring-v1 README, and this paragraph is its first language-neutral statement (Section 6). The SHA-256 of these bytes is the ring's identity. The chain field `prev_ring_sha256` is the SHA-256 of the previous month's ring file bytes. The reference builder computes it over the canonical bytes of the loaded previous ring and refuses a previous ring whose file bytes differ from them, so file bytes and canonical bytes are the same thing by construction; an earlier version hashed a compact re-serialization instead, a defect found and fixed before Ring 001 was anchored.

3.4 Anchoring

A month's rings are listed in a plain text file of SHA-256 values with paths (rings/2026-08.sha256). That file's SHA-256 is appended to the JIDEC ledger as a claim, and the claim is stamped with OpenTimestamps, which places it in a Bitcoin block within hours. Ring 001, covering August 2026 for eight endpoints, is JIDEC entry 32 (claim SHA-256 `f3e589efca103f3f717a68857618411f5f0864e0ad1aa264089e70b9d89081cc`, confirmed in Bitcoin block 965566 on 2026-09-05 04:53 UTC).

4. Protocol

4.1 What “anyone can recompute” means operationally

1. Obtain the history export for the endpoint and month as committed in the public repository.
2. Run a builder that implements Section 3.
3. Compare the resulting bytes, or their SHA-256, with the anchored value.

The reference builder exposes this as a verify mode: given a ring file and the history it claims to be built from, rebuild and compare byte for byte, printing the first differing field if any.

4.2 Independent reimplementation

The stronger test adds four conditions. The second implementer (a) has not read the body of the reference source; (b) reads the specification and confirms by SHA-256 that it is the document the reference cites; (c) implements in a different language, so that runtime defaults cannot be inherited by accident; (d) runs the byte comparison blind, in the sense narrowed below, and reports per-ring match or mismatch before either implementer has read the other’s account of the result.

Condition (a) is a statement by the implementer and cannot be proven; it is recorded as such in the anchored record (“implementation_2_source_seen: none of implementation_1 (implementer’s statement)”). The claim needs to be narrower than “never touched in any form.” In an earlier, separate recompute, the reference builder was run as a black box (`make_ring.py --verify` was executed against the committed history exports and its output compared against the eight published rings) and its header was opened far enough to read the specification hash it cites, the same value checked under condition (b) below. Neither of those exposed the file’s algorithm. What was not read, before or during the reimplementation, was the body of `make_ring.py`: the logic that turns history entries into ring fields. That narrower claim is the one this paper stands on. The anchored record is not edited; the narrowing is appended to the same ledger as a correction record, JIDEC entry 36, and both records are reproduced in Appendix B.

Condition (b) is checkable and was checked: the specification’s SHA-256, `9ccba2e325fd2a555fdb2dec519b8c6bf7a669064674846aea98ecff824e3d`, matches the value cited at the head of the reference builder, the same header read under condition (a) above, not a second, independent citation. It was computed by fetching `NENRIN_SPEC_v1.md` fresh and hashing the response, so that a stale or substituted copy of the specification could not silently pass against the wrong document.

Condition (c) holds structurally, not just nominally: the reference is Python, the reimplementation is Node.js, and the two share no runtime, no standard library, and no default serialization behaviour. This is what exposed the seam reported in Section 6: a shared runtime could have inherited the reference’s key-ordering behaviour without either implementer noticing that the specification had never stated it.

Condition (d) needs a correction too, and it is a more useful one than the claim it replaces. The reimplementation was not built from `NENRIN_SPEC_v1.md`’s Layer 3 section alone: that section sketches a ring’s shape but names only a subset of its twenty fields; `first_instant`, `last_instant`, `instants_by_status`, `instants_by_consent_source`, `record_sha256_first`, `record_sha256_last`, `witness_identities`, `prev_ring`, and `recompute` are not in the spec’s own text. The remainder of the field set, and its exact shape, came from the published ring files themselves: real, already-anchored examples fetched from the committed repository, read before the reimplementation’s own code was written. “Runs blind against the published inputs” needs the same narrowing: the shape of a ring’s output had already been seen. What stayed blind, and what the verify mode actually tested, was the comparison: the reimplementation’s rebuilt bytes for all nine fresh history exports were never checked against the eight claimed ring files until the run reported match or mismatch, field by field, before either implementer’s account of the other’s result was read.

4.3 Recording the result

The result is not a message; it is a record. The second implementer’s file was fetched fresh from its public repository at a named commit and hashed. The reference builder, the specification, each history export, and each anchored ring hash were hashed on the spot. All of these, with the result and its limits, were serialized as one plain text record and appended to the ledger as JIDEC entry 34 (claim SHA-256 `fad6d00a25281102711573b151b321bc13b28c625fe65807c5eb3a12a04e393c`, confirmed in Bitcoin block 965627 on 2026-09-05 14:58 UTC). The second implementer then independently fetched entry 34 raw and re-hashed it, and fetched his own file from GitHub at the commit and re-hashed it, confirming both values. No hash in this paper was copied from a message.

5. Results

5.1 The eight August rings

Table 1 lists the eight endpoints, the counts their rings carry, and the outcome of both implementations. All eight endpoints are

operated by the first author (Layer 4, self-application); no external endpoint had consented to tool calls in August, which is itself stated in every ring’s limits.

Endpoint (slug)	sampled	reached	by status	manifest hashes	witnesses	ring SHA-256 (prefix)	Python	Node.js
mcp-horizonshield-dev-mcp	26	26	verified 23, pending 3	3	1	9ca61125	match	match
hearing-horizonshield-dev-mcp	26	26	verified 23, pending 3	2	1	ceb47d33	match	match
web-horizonshield-dev-mcp	26	26	verified 23, pending 3	1	1	06d1f77b	match	match
gate-horizonshield-dev-mcp	26	18	verified 17, held 9	3	1	e473f717	match	match
jidec-horizonshield-dev-mcp	24	22	verified 22, held 2	1	1	3a215b6c	match	match
p001-horizonshield-dev-mcp	26	26	pending 26	1	1	00155986	match	match
p002-horizonshield-dev-mcp	26	26	verified 18, pending 8	1	1	1f8d4f45	match	match
femtech-horizonshield-dev-mcp	5	5	verified 5	2	1	7a310f5d	match	match

Two rows deserve comment because they are unflattering and were kept. The gate row is the instrument measuring itself: nine of twenty-six instants are held because the gate cannot reach its own door from inside the same edge network, a limitation the ring states rather than hides. The p001 row is a partner endpoint that is pending on all twenty-six instants; a ring that could only be built when the numbers were good would not be a record. Every one of those twenty-six is pending for the same reason: the owner had not consented to a tool call, so determinism was not measured, and the gate’s status label reads pending for an unmeasured condition exactly as it does for a failed one. The record behind each instant keeps the two apart (measured: false); the label, and the ring’s instants_by_status count, do not. Section 7 returns to this.

5.2 Artifacts

Table 2 lists every artifact whose hash appears in entry 34.

Artifact	Identity
Specification, NENRIN_SPEC_v1.md	SHA-256 9ccba2e325fd2a555fcd2dec519b8c6bf7a669064674846aea98ecff824e3d
Reference builder, make_ring.py (Python)	SHA-256 69719fed5ae6387bc9b363914e61ab70c8bfee320710fcd028191b90e41aa2c4
Independent builder, make_ring.js (Node.js)	SHA-256 5167188aeefb4852ca941a96856724f8831abd46be331cd9544898ba038e82a8, 7,955 bytes, github.com/babyblueviper1/invinoveritas, commit 917dd97ff8e30107810d9a059e9091077f5171d0 (2026-09-05T13:08:01Z)
History exports (9 files, committed)	SHA-256 of each listed in entry 34
Ring hashes, rings/2026-08.sha256	JIDEC entry 32, claim f3e589ef...
Result record	JIDEC entry 34, claim fad6d00a..., schema nenrin-ring-reimpl-match-v1, Bitcoin block 965627
Correction record	JIDEC entry 36, claim 69158463e659d3b3d158d8068a0506ccc9101beeca6ab2acc5b87aec493edc9c, schema nenrin-ring-reimpl-match-v1-correction; narrows two provenance phrases of entry 34, changes no hash and no result

The result: eight of eight rings byte-identical across the two implementations, from the same committed history, with the byte comparison performed blind.

6. The seam

The reimplementation succeeded, and the reason it did is the more useful result: one behaviour was checked rather than assumed, and the check is what closed the gap.

Python’s `json.dumps(sort_keys=True)` sorts dictionary keys at every level of nesting. JavaScript’s `JSON.stringify` sorts nothing; it emits object keys in insertion order. A port that reproduced the ring-building logic exactly and serialized with `JSON.stringify(obj, null, 2)` directly would have produced output that looks like a ring, parses as a ring, carries the same counts, and hashes to a different value on any object whose insertion order differs from sorted order, which, for this schema, is most of them. Neither implementation alone can detect this: each is internally self-consistent, and a single implementer re-running their own code will get their own answer back every time. Only a second, independent implementation exposes it, and only if that implementation is compared byte for byte rather than field for field. The reimplementer adds an explicit recursive key sort (every plain object’s keys sorted, every array’s own order left untouched) immediately before serialization, applied once at the top of the ring object so it reaches every nested level the way Python’s single flag does in one pass. That one function is the entire fix; everything else in the builder can be wrong in some unrelated way and this would not catch it, which is a limit of the check as much as a description of it.

The seam belongs to the specification, not to either implementer, and the gap is wider than “names a language instead of stating rules.” `NENRIN_SPEC_v1.md` does not state the canonical form at all: no `json.dumps`, no `sort`, no mention of canonicalization anywhere in it. The rule lives in the reference builder’s own docstring and the ring-v1 `README`, which are artifacts, not the specification. Section 3.3 of this paper, stating the rule as sorted keys at every level, two-space indentation, raw non-ASCII, and one trailing newline, independent of any language, is its first language-neutral statement. The specification did not under-specify the canonical form so much as omit it, and the artifacts carried it instead. That gap was reproducible in practice, because both implementers independently arrived at the same artifacts and the same effective ruleset, but it was invisible in principle: nothing in the document a reimplementer is told to build from says a canonical form exists to find. RFC 8785 exists for exactly this: to state key ordering, string escaping, number formatting, and whitespace independently of any runtime’s defaults, so that “canonical” describes the bytes rather than a call site.

Adopting it would not be neutral between the two implementations reported here. RFC 8785 Section 3.2.3 specifies sorting property names as arrays of UTF-16 code units, compared as unsigned integers, and states the reason plainly: this maps directly to the native string type in ECMAScript, not to Python’s. On the key-collation axis of Table 3, the Node.js reimplementer’s default behaviour is already RFC-8785-conformant by construction. The Python reference, whose `sort_keys=True` orders by Unicode code point, is the implementation that would have to change if a future revision of the specification adopted RFC 8785 as written, not the other way around, for any endpoint whose status or consent-source values ever leave the ASCII range.

Table 3 lists the latent seams of this class and reports, from the reimplementer’s own code, which are exercised by the August rings and which are not.

Seam	Python reference behaviour	Node.js reimplementer behaviour	Exercised by August rings?	Risk
Nested key order	sorted at every level	explicit recursive sort of every object’s keys, applied once at the ring’s root before serialization	yes, every ring	found and closed
Key sort collation	by Unicode code point, which is not the RFC 8785 order	<code>Array.prototype.sort()</code> with no comparator, i.e. by UTF-16 code unit, which is the order RFC 8785 Section 3.2.3 specifies	no; every fixed schema field name is ASCII, and the only <i>data-derived</i> keys in the schema (<code>instants_by_status</code> , <code>instants_by_consent_source</code> , keyed by each entry’s own <code>status/consent_source</code> string) were ASCII in every August entry	latent, and narrower than it first looks: the risk lives specifically in those two data-derived key sets, not in the schema’s own field names, which are fixed and known ahead of time. A non-ASCII status or consent-source label would be the first thing to exercise it, not an exotic input. If a revision adopts RFC 8785 as written, the Python reference is the implementation that changes, not the Node.js one
String escaping of control characters	<code>\uXXXX</code> lowercase	<code>JSON.stringify</code> ’s built-in escaping, not independently implemented	no (no control characters in ring values)	latent; not independently verified against the Python reference’s exact escaping, since none of the eight rings’ values contain a control character to force the comparison
Non-ASCII characters	emitted raw (<code>ensure_ascii=False</code>)	emitted raw; <code>JSON.stringify</code> has no ASCII-escaping mode to opt out of, and the output is written via <code>Buffer.from(..., "utf-8")</code>	no (all August ring values are ASCII)	latent; both runtimes emit raw UTF-8 by construction here, so a match is expected but has not been forced by real data
Number formatting	integers only	plain JavaScript numbers, all integer-valued in this schema (counts and lengths only)	yes	none while every number is a count; a float anywhere in the schema would open this row for real
Indentation and separators	two spaces, <code>\n</code> and <code>:</code>	<code>JSON.stringify(obj, null, 2)</code> , empirically identical formatting to Python’s <code>indent=2</code> for the value types this schema uses	yes, every ring	matched
Trailing newline	one	one, appended explicitly outside <code>JSON.stringify</code>	yes, every ring	matched
Chain hash input	sha256 of the loaded previous ring’s canonical	<code>prev_ring_sha256</code> computed by hashing the reimplementer’s own	not yet; August 2026 is the first ring, <code>prev_ring</code> is null,	the code path exists and was written to the same rule as the reference, but “written correctly” and

	bytes; the loader refuses a previous ring whose file bytes differ from them, so this equals the file bytes by construction	canonical bytes of the previous ring object, mirroring the same rule	so this path in the reimplementation has never actually run against real data	“exercised” are different claims; this row is unverified, not verified-and-latent, until a real chain hash is computed and compared
--	--	--	---	---

Two rows deserve a sharper statement than “not yet exercised.” The chain-hash row is not a risk the reimplementation carries and the August test simply didn’t need to run; it is a piece of the reimplementation that has never been run against real data at all, in either language, and the two implementers’ code could disagree there in a way neither of the eight matches reported in Table 1 would have caught. The witness path (Section 7) is the same kind of gap for a different reason: the schema has a place for more than one witness, and nothing about producing a single-witness ring correctly demonstrates that a multi-witness ring is built the same way twice. Both are exactly what the September rings, pre-registered in Section 8, exist to test.

7. Limits

This paper claims less than it might appear to, and the record it cites says so in its own text.

Determinism is not truth. A byte-identical ring proves that the ring layer is a deterministic function of its inputs. It proves nothing about whether the inputs, the measurements inside the history export, are accurate. That question belongs to Layers 1 and 2 (open witnessing and discrepancy records), where an independent party measures the same endpoint and disagreement becomes a citable record. In August 2026 every ring carries witnesses: 1, the operator’s own instrument, and the limits sentence in every ring says that no discrepancy could have been recorded because nobody else was positioned to record one.

Two is not many. Two implementations by two cooperating people is the minimum that makes the word “independent” meaningful, and no more. The second implementer is a technical peer who has cross-verified other parts of this ledger; he is not an adversary. An adversarial reimplementation, one that tries to produce a ring that parses and passes casual inspection while hashing differently, is a different test and has not been run.

The inputs are operator-published. The history exports were committed by the operator. The second implementer fetched them from the operator’s repository. Reproducing the operator’s rings from the operator’s exports is a test of the builder, not of the operator. The anchoring of the exports’ hashes in entry 34, and the daily append-only archive, make later alteration detectable; they do not make the original measurement independent. The independent measurement exists (one witness walk of the gate endpoint on 2026-09-05, submitted to the ledger by the second implementer) and enters the record in the September rings.

The canonical form is not in the specification. At version 1 it is stated only by the reference builder and its README, and it reached the second implementer through the published ring files; Section 6. The next revision must state it, and if it adopts RFC 8785 as written, the Python reference is the implementation that changes.

The status label collapses two distinctions that the records keep. A gate record separates an instrument failure (reachable null, gate_side true) from a target that did not answer (reachable false), and a condition that was not measured (measured false) from one that was measured and failed; the one-word status does not, reading held for the first pair and pending for the second, and the ring’s instants_by_status inherits the label. The unmeasured count survives in the ring only as prose in its limits sentence. Both collapses were found by the first author while writing an adapter for an open semantic type system for verification evidence (trustless-ai/semantic-abi), where they appear as test vectors that fail against his own system and are filed as such. The fix belongs to the next ring schema: counted fields for instrument failures and for unmeasured conditions, beside the status counts.

The retention failure happened. Section 3.1. A reproducibility claim that depends on a live cache is not a reproducibility claim; the fix was to commit the inputs, and it was made one sweep before it would have been too late.

8. Pre-registration of the next test

The September 2026 rings will be built in the first days of October 2026 from the committed history exports and the witness records accepted by the ledger during September. Two things will be exercised for the first time: the chain field (prev_ring_sha256 equal to the SHA-256 of the August ring file bytes) and the witness path (the gate endpoint will carry witnesses: 2, and its limits sentence will change accordingly). The second implementer has stated that he will rerun make_ring.js against the September inputs and report per-ring match or mismatch before either implementer has read the other’s result, the same sense of blind as in Section 4.2(d). The outcome, either way, will be appended to the ledger and reported in a revision of this paper. A mismatch on the chain or witness path would be a more valuable result than the eight matches reported here, because it would locate a second seam.

9. Conclusion

“Anyone can recompute this” was true in principle from the day the specification was anchored and untested in fact until someone

who had not read the source did it in another language. The eight matches are the evidence. The seam is the finding. The limits are the honest perimeter: this shows that the ring layer is a deterministic public function, and it points to the exact places, the chain and the witness path, where the next test will look.

Same file in, same bytes out, from someone who had not read the source. That is what verifiable was supposed to mean, and now, for one layer of one system, it has been checked rather than claimed.

Appendix A. Reproduce it yourself

Clone github.com/ogasurfproject-jpg/mcp-conduct-register. For any August ring:

```
python3 scripts/make_ring.py --verify rings/<slug>/2026-08.json --history history/<slug>.json
```

Compare the printed SHA-256 with the corresponding line of `rings/2026-08.sha256`, and compare the SHA-256 of that file with the claim of JIDEC entry 32 at `ledger.horizonshield.dev/ledger/32`. For the independent implementation, clone github.com/babyblueviper1/invinoveritas at commit 917dd97f and run `scripts/nenrin_ring_reimpl/make_ring.js` against the same history files.

Appendix B. The anchored result record

JIDEC entry 34, fetched raw from `ledger.horizonshield.dev/ledger/34?format=raw`. SHA-256 of these exact bytes: `fad6d00a25281102711573b151b321bc13b28c625fe65807c5eb3a12a04e393c`. Anchored in Bitcoin block 965627 (2026-09-05 14:58 UTC). Reproduced unedited, including the surname without its accent, as the record was written.

```
schema: nenrin-ring-reimpl-match-v1
date: 2026-09-05
claim: a second, independent implementation of NENRIN Layer 3 (ring builder), written from the spec alone in a different language by a different person, reproduces all eight August 2026 rings byte by byte from the same history exports
spec: NENRIN_SPEC v1.md sha256 9ccba2e325fd2a555fcd2dec519b8c6bf7a669064674846aea98ecfff824e3d (Layer 3)
implementation_1: python, make_ring.py sha256 69719fed5ae6387bc9b363914e61ab70c8bfee320710fcd028191b90e41aa2c4, author Toshikatsu Oga, horizon-shield workers/hs-ledger/nenrin/ring-v1
implementation_2: node.js, make_ring.js sha256 5167188aeefb4852ca941a96856724f8831abd46be331cd9544898ba038e82a8, 79 55 bytes, author Federico Blanco Sanchez-Llanos, github.com/babyblueviper1/invinoveritas scripts/nenrin_ring_reimpl/make_ring.js commit 917dd97ff8e30107810d9a059e9091077f5171d0 (2026-09-05T13:08:01Z)
implementation_2_source_seen: none of implementation_1 (implementer's statement)
inputs: history exports as committed in github.com/ogasurfproject-jpg/mcp-conduct-register
  history femtech-horizonshield-dev-mcp.json sha256 9e5098fc8825c8870031d666180ce86b562ce39ab32043ede45dfefb5e79ed64
  history gate-horizonshield-dev-mcp.json sha256 dafef6a0d6e7d93df7183e9809c7e228fca41f3784d43d2409a4cabd7c7e0841
  history hearing-horizonshield-dev-mcp.json sha256 d6584dc3671a65509af95e7949294636d11668e23af6a4ad6f9816ef9f566759
  history intel-twzrd-xyz-mcp.json sha256 7919c0bf674300cc80ed5bf167fc6fcd9d06673ef2fd27203cca7392a31b1051
  history jidec-horizonshield-dev-mcp.json sha256 18b6a8b128a0248d92be3109237ade5e894906162a853e8dddc7128614a02761
  history mcp-horizonshield-dev-mcp.json sha256 21420f279a3ac9183fa701ac16ad86e7d1e40bbe2027439304bf7fea9b1758bd
  history p001-horizonshield-dev-mcp.json sha256 28495507bc5c3c9d4dfe6c3d172221e28b0586922fb171e2b455f3199591454c
  history p002-horizonshield-dev-mcp.json sha256 c92271834ee9a04fd6fca9978aa52f00062deac5e66f3687d3986deafb5ea335
  history web-horizonshield-dev-mcp.json sha256 513fea9029699ad911a61db88fb8b79555584c6aa394f3f0fe0a0513528bf79
outputs expected: rings/2026-08.sha256 (JIDEC entry 32)
  7a310f5dd1eed7b2bbf8a9a8226539db6b33bad43bfae645742018e566f8376 rings/femtech-horizonshield-dev-mcp/2026-08.json
  e473f71705e5e440ba2c0197545df04f88926314208c2dfc915c1b40b5dd440b rings/gate-horizonshield-dev-mcp/2026-08.json
  ceb47d3366ca9174db8f7536c26f9d5b8d19f027a8ee258d6e22a1fefea77436 rings/hearing-horizonshield-dev-mcp/2026-08.json
  3a215b6ceb4cae71b7d44b29927ba49d19293d74f84369ea9269f05c57a0a075 rings/jidec-horizonshield-dev-mcp/2026-08.json
  9ca61125dcb3e566a34e4542641324987b1fa2a3f1f5f8c5086126f45aa4b953 rings/mcp-horizonshield-dev-mcp/2026-08.json
  00155986d417310807c3da85f3b0d7f4640c899b298db89a27a6e82af2f4d1b4 rings/p001-horizonshield-dev-mcp/2026-08.json
  1f8d4f452dd3c4c95d4c24e63bb4fb603f272b13ce04b9348b1d0f0040bb7602 rings/p002-horizonshield-dev-mcp/2026-08.json
  06d1f77bdd96b1b8d3fb0496c1fc1ff307aae94cd75f148d3037c98b1b1086e8 rings/web-horizonshield-dev-mcp/2026-08.json
result: 8 of 8 byte-identical (implementer's run, 2026-09-05; the eight sha256 values he reported equal the eight above)
implementation note: python json.dumps(sort keys=True) sorts object keys at every nesting level; JSON.stringify sorts none, so implementation_2 applies a recursive key sort (array order preserved) before serialising with 2-space indent and a trailing newline
limits: validates determinism of Layer 3 only; says nothing about the truth of the measurements inside history; n=2 implementations; August rings carry one witness; the witness-record path (witnesses >= 2) is first exercised by the September rings and is not covered by this record
supersedes: the sentence in mcp-conduct-register README 'a from-scratch reimplementation in another language ... has not yet been done' (2026-09-05, earlier the same day)
```

Two phrases in this record, “written from the spec alone” and “implementation_2_source_seen: none of implementation_1”, are broader than the account in Section 4.2: the reference builder was executed as a black box in an earlier recompute, and the field set and canonical form were taken from the published ring files. An anchored record is not edited. The narrowing is appended instead,

in the register's README (2026-09-06) and as a correction record on the same ledger, JIDEC entry 36.

JIDEC entry 36, fetched raw from ledger.horizonshield.dev/ledger/36?format=raw. SHA-256 of these exact bytes: 69158463e659d3b3d158d8068a0506ccc9101beeca6ab2acc5b87aec493edc9c. OpenTimestamps proof submitted on 2026-09-06 and pending at the time of writing; the Bitcoin block appears in the ledger entry when confirmed.

```
schema: nenrin-ring-reimpl-match-v1-correction
date: 2026-09-06
corrects: JIDEC entry 34, claim sha256 fad6d00a25281102711573b151b321bc13b28c625fe65807c5eb3a12a04e393c, schema nenrin-ring-reimpl-match-v1, Bitcoin block 965627
method: the corrected entry is not edited; this record is appended and cites it, and both stay readable
phrase 1: "written from the spec alone" (the claim line of entry 34)
narrowed 1: the specification's Layer 3 sketch names 11 of a ring's 20 fields and does not state the canonical form; the remaining fields (first instant, last instant, instants_by_status, instants_by_consent_source, record_sha256_first, record_sha256_last, witness_identities, prev_ring, recompute) and the byte rules were taken by the implementer from the published ring files in mcp-conduct-register, read before the reimplementation was written
phrase 2: "implementation_2_source_seen: none of implementation_1 (implementer's statement)"
narrowed 2: in a recompute earlier the same day (reported 17:49 JST), the implementer cloned mcp-conduct-register, executed scripts/make_ring.py --verify against the eight rings as a black box, and read its docstring header for the specification hash it cites; the body of make_ring.py, the logic that turns history entries into ring fields, was not read before or during the reimplementation (implementer's statement)
what_was_blind: the byte comparison; the rebuilt bytes were not checked against the eight published rings until the verify run reported match or mismatch
what_stays: the eight byte-identical rings, their eight sha256 values, the specification and implementation hashes, and the nested key-order seam; none of these depends on the two phrases
source_of_narrowing: the implementer's own account in Section 4.2 of papers/nenrin-reproducibility/manuscript_v0.1.md, github.com/ogasurfproject-jpg/horizon-shield, his commit 7dc5bff8, on main by merge e51feb96, written after the operator's review found the two phrases broader than the record supports
also_appended: mcp-conduct-register README, 2026-09-06 correction line under the 2026-09-05 update
limits: this record narrows two statements of provenance; it changes no hash and no result
```

Acknowledgements and disclosure

The first author operates every endpoint measured in this paper and the ledger that anchors the records; that is Layer 4 of the design, and it is also a conflict of interest, stated here. The second author operates invinoveritas, a separate verification service; none of its endpoints is in the rings reported here, and it has been measured under the same ledger's witness path. The two authors are also collaborating on a separate open project, an adapter for the semantic-abi type system, and no payment has passed between them in either direction. Neither author received compensation for any part of this work. No party paid for any measurement or record cited.

References

- Laurie, B., Langley, A., Kasper, E. (2013). Certificate Transparency. RFC 6962, IETF. <https://www.rfc-editor.org/rfc/rfc6962>
- Laurie, B., Messeri, E., Stradling, R. (2021). Certificate Transparency Version 2.0. RFC 9162, IETF. <https://www.rfc-editor.org/rfc/rfc9162>
- Sigstore project. Rekor: a transparency log for software supply chain signatures. <https://github.com/sigstore/rekor> and <https://docs.sigstore.dev>
- Reproducible Builds project. <https://reproducible-builds.org>
- Lamb, C., Zacchiroli, S. (2022). Reproducible Builds: Increasing the Integrity of Software Supply Chains. IEEE Software 39(2), 62 to 70. <https://doi.org/10.1109/MS.2021.3073045>
- Debian Project. ReproducibleBuilds. <https://wiki.debian.org/ReproducibleBuilds>
- Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R., Cappos, J. (2019). in-toto: Providing farm-to-table guarantees for bits and bytes. Proceedings of the 28th USENIX Security Symposium, 1393 to 1410.
- in-toto Attestation Framework, Statement v1. <https://github.com/in-toto/attestation>
- SLSA: Supply-chain Levels for Software Artifacts, version 1.0 (2023). <https://slsa.dev>
- Todd, P. (2016). OpenTimestamps: Scalable, Trust-Minimized, Distributed Timestamping with Bitcoin. <https://petertodd.org/2016/opentimestamps-announcement> and <https://opentimestamps.org>
- Rundgren, A., Jordan, B., Erdtman, S. (2020). JSON Canonicalization Scheme (JCS). RFC 8785, IETF. <https://www.rfc-editor.org/rfc/rfc8785>
- Model Context Protocol specification. <https://modelcontextprotocol.io/specification>
- Oga, T. (2026). NENRIN v1: Machine-Readable Tree Rings for Agent-Facing Services (NENRIN_SPEC_v1.md), SHA-256

9ccba2e325fd2a555fcd2dec519b8c6bf7a669064674846aea98ecff824e3d. github.com/ogasurfproject-jpg/horizon-shield,
workers/hs-ledger/nenrin/. Reference builder make_ring.py, SHA-256
69719fed5ae6387bc9b363914e61ab70c8bfee320710fcd028191b90e41aa2c4, in ring-v1/.

Oga, T. (2026). JIDEC_PATH_SPEC_v1.md, the jidec-path-v1 record format (JIDEC ledger entry 5). github.com/ogasurfproject-jpg/horizon-shield, workers/hs-ledger/.

Blanco Sánchez-Llanos, F. (2026). make_ring.js, independent Node.js implementation of NENRIN Layer 3.
github.com/babyblueviper1/invinoveritas, scripts/nenrin_ring_reimpl/make_ring.js, commit
917dd97ff8e30107810d9a059e9091077f5171d0, SHA-256
5167188aeefb4852ca941a96856724f8831abd46be331cd9544898ba038e82a8.

JIDEC ledger, HORIZON SHIELD. Entry 32 (Ring 001, rings/2026-08.sha256, claim
f3e589efca103f3f717a68857618411f5f0864e0ad1aa264089e70b9d89081cc), entry 33 (instant coordinate addendum), entry 34
(reimplementation match record, claim fad6d00a25281102711573b151b321bc13b28c625fe65807c5eb3a12a04e393c, Bitcoin block
965627), entry 36 (correction record narrowing two phrases of entry 34). <https://ledger.horizonshield.dev/ledger/{n}> and ?
format=raw for the exact bytes.

mcp-conduct-register. Public register with committed history exports and rings. github.com/ogasurfproject-jpg/mcp-conduct-register

Oga, T. (2026). Verification for the Buyer, Not the Seller. SSRN working paper 6964439. <https://ssrn.com/abstract=6964439>

Oga, T. (2026). A Demand-Side Benchmark for Consumer-Facing Construction Cost Questions: Price-Figure Span, Output Consistency, and the Case for a Verifiable Reference Layer. engrXiv 7814. <https://doi.org/10.31224/7814>

Oga, T. (2026). SSRN working paper 6872819 (a benchmark of general-purpose language models against a structured construction-cost engine on renovation cost questions). <https://ssrn.com/abstract=6872819>